

Minsoo Kim



+8210-7714-9201



vibrio0102@gmail.com



[linkedin](#)



[github](#)

Core Skills

Requirements discovery and stack selection: Learned a new medical domain and interviewed researchers to identify bottlenecks in data production and sharing. Defined component responsibilities and designed and built an in-house MLOps platform from scratch.

Data pipelines and reproducibility: Connected ingestion, transformation, versions and lineage with Airflow, dbt, Iceberg and DataHub. Built shared query access using the SeaweedFS REST Catalog and PostgreSQL + pg_duckdb, plus manifest freeze/restore tools and MLflow experiment and model management.

On-premises infrastructure and hardware operations: Sole owner of bare-metal Kubernetes cluster construction, managed through 12 Pulumi stacks. Led RAM/NVMe reuse, network-equipment configuration and NUMA, memory and disk adjustments.

Cloud data engineering and collaboration: Built AWS EMR/PySpark pipelines and vector-search workflows, and led a data engineering team. Apply experience across iOS, backend and frontend development to define practical ownership boundaries between researchers and the platform team.

Technical Skills

Languages: Python | Swift | JavaScript | Objective-C | Shell | SQL

Frameworks: PySpark | Flask | FastAPI | Kafka | Langchain | OpenAI | ReactJS | NextJS | Bun

laC & Platform: Pulumi | Kubernetes(kubeadm) | Cilium | SeaweedFS | Dex/OIDC | Docker

Data: Airflow | dbt(Cosmos) | DataHub

Lakehouse & OLAP: Apache Iceberg | DuckDB | ClickHouse

Database: MariaDB | PostgreSQL | PGVector | Firestore | Neo4J

ML: Ray/KubeRay | MLflow

Observability: Prometheus | Grafana | Loki | Vector

LLM: Langchain | LangGraph | OpenAI(Text Embedding) | RAG(Neo4J, PostgreSQL)

Data Analytics: CytoscapeJS | SigmaJS

iOS: ReactorKit | Async Await | Alamofire | Snapkit | Crashlytics | Kingfisher | SceneKit | SPM | React-Native

Cloud: AWS (EC2, Lambda, S3, IAM, EMR, Route53, RDS Aurora) | Firebase (Firestore, Storage) | GCP | Ubuntu | uv

Career Summary

Data Engineer (approx. 3 years): Designed and built an on-premises MLOps platform and operated data pipelines on AWS. Led data-platform and MLOps pipeline work with PySpark, Airflow and vector databases.

Backend Engineer (cumulative approx. 2-3 years): Built API servers and internal management systems in Python (Flask, FastAPI), including real-time WebSocket communication and Firebase integration.

Frontend & Client (approx. 4 years): Built data-visualization dashboards and BI tools with Next.js. Started out as an iOS developer, designing ReactorKit-based reactive architectures and shipping native apps.

Experiences

HUINNO

Data Engineer

Dec 2025 - Present

Seoul

- **Led requirements discovery and greenfield MLOps delivery:** Interviewed researchers and examined data, code and execution workflows to identify producer dependencies, scattered datasets and missing versions and lineage. Assigned clear responsibilities to Airflow for execution and retries, dbt for SQL transformations, Iceberg for table versions and DataHub for lineage, then designed and implemented the shared platform.

- **Built and operated on-premises bare-metal Kubernetes:** Sole owner of cluster construction on a two-person platform team. Built a 10-node cluster from company hardware and codified networking, storage, identity and execution environments in 12 Pulumi stacks (as of September 2026). Tracked infrastructure changes and inter-stack dependencies to support repeatable deployment.
- **Led hardware configuration and operations:** Disassembled idle PCs and workstations to reclaim and reinstall RAM and NVMe drives; selected and replaced network equipment. Addressed NUMA and dual-channel configuration, memory pressure and swap, and separated the PostgreSQL data disk, adapting both hardware and system settings to actual workloads.
- **Turned tacit data-production knowledge into shared pipelines:** Data was scattered across nodes, generation code was unversioned and access depended on a few producers, making datasets difficult to obtain in their absence. Replaced SSH-based file transfers with shared storage and Airflow/dbt Bronze/Silver/Gold pipelines with lineage in DataHub. Implemented the common framework, research-data pipelines, MIMIC-IV ingestion and operational ETL.
- **Designed Iceberg catalog and DuckDB query access:** Used SeaweedFS S3-compatible storage and its Iceberg REST Catalog to manage Silver/Gold data and metadata with NVMe/HDD storage tiers. Built PostgreSQL + pg_duckdb query access. For dimension-table generation, resolved the latest metadata location from the catalog and passed it to DuckDB, avoiding version inference from filenames.
- **Built data-version and experiment reproducibility foundations:** Models and performance were shared in Notion without linked records of data, generation code and execution environments. Implemented manifest freeze/restore tools and validation procedures for data versions, code commits and environment metadata; deployed MLflow tracking and a model registry with SeaweedFS artifact storage. Research-code integration and end-to-end data-to-model reproducibility validation remain in progress.
- **Established operational checks and ownership boundaries:** Implemented recovery and integrity checks for missing raw data and separated services and buckets by environment. Defined researcher ownership of domain ETL and platform ownership of execution, lineage, validation and retries; refined rollout plans through follow-up interviews. Connected Dex/OIDC identity to Kubernetes RBAC and built a shared KubeRay training environment to support research-code migration.

Cellkey

Jul 2025 - Dec 2025

Manager, Data Engineer

Seocho-gu, Seoul

- Designed a data-cleaning pipeline on **Apache Airflow 3.x** and deployed it locally.
- Maintained and tuned **AWS Lambda, Batch and Step Functions** to carry scalable data workflows.

Netmelon

Mar 2025 - Jun 2025

Back-End Engineer

Freelance

- Implemented Android & iOS In-App Purchase API (subscriptions, renewals, cancellations)
- Packaged a Flask web server as a Docker image and deployed it on Google Cloud Run
- Implemented a feature to verify ownership of Firebase resources
- Wrote unit test cases for LLM-driven API routes, services, and functions
- Operated Firebase Firestore and Cloud Storage
- Refactored legacy code

Eazel Inc.

Jun 2022 - Sep 2024

Team Lead, AI Labs

Jung-gu, Seoul

- Owned the art-market ETL pipeline as team lead — architecture, deployment, monitoring and performance tuning
- Built dashboards based on Flask/NextJS
- Developed and operated auction data scrapers
- Built Airflow pipelines that collect, clean and load data on a schedule
- Built cleaning and loading pipelines with PySpark on AWS EMR
- Built a deduplication system on OpenAI and Facebook embedding models, then raised its accuracy and speed with a vector DB
- Managed AWS services (EC2, IAM, S3, Route53, RDS, EMR)
- Implemented CI/CD using Github Actions
- Built a chatbot prototype with LangChain and Streamlit
- Led the implementation of a Neo4j Cypher query tool and a PostgreSQL query tool for the chatbot
- Built the scraping layer on LangChain with information-extraction prompts

Eazel Inc. MLOps Engineer, MLOps	Oct 2021 - Jun 2022 Jung-gu, Seoul
• Ran the full cycle from data analysis through modeling to validation, using HuggingFace NLP among other tooling • Developed and operated Instagram data scrapers • Built a social-network analysis dashboard on Flask / NextJS / CytoscapeJS and surfaced the findings that fed a Series A raise • Wired the ML lifecycle through DVC • Managed the data in PostgreSQL	
Eazel Inc. iOS Developer, iOS Team	Jan 2019 - Oct 2021 Gangnam, Seoul
• Led the design of the API client and its backend integration around iOS app performance and scalability • Introduced CI/CD with Github Actions • Introduced ReactorKit-based reactive programming and led the architecture behind it, improving app responsiveness and maintainability	
Finple Inc. Software Engineer, DevOps	Jan 2018 - Nov 2018 Yeongdeungpo, Seoul
• Maintained hybrid iOS and Android apps • Developed a NodeJS-based data scraper	

Projects

Eazel Inc. Data Engineer, AI Labs	Jun 2022 - Sep 2024 Jung-gu, Seoul
Development and Operation of Data Pipeline for an Art-Market Analytics Service	Jan 2023 - Oct 2023
• Built the pipeline on one AWS Aurora RDS to keep development cost down. • Processed and loaded data with PySpark on AWS EMR. • Implemented partitioning and data optimization for distributed processing. • Key contribution to Series B funding by leading the overall design and construction of the data platform (analytics/MLOps).	
Development and Operation of Art Market Data Scraper	Jan 2023 - Feb 2023
• Built a scraper with requests and BeautifulSoup4. • Built a fast scraper on multithreading. • Used LangChain information-extraction prompts to scrape into a fixed structure. • Developed a social relationship collection scraper written in Swift, operating in the iOS environment. • Ran 50 scraper instances at once through boto3. • Controlled those instances over a WebSocket link to the management server.	
Development and Operation of Central Management Server for Data Scrapers	Jan 2023 - Feb 2023
• Built Flask endpoints that manage every scraper from one place. • Pushed each scraper its targets over that WebSocket connection. • Hashed login credentials with SHA-256 in transit. • Split the endpoints into Flask blueprints. • Put Nginx in front as a reverse proxy. • Managed HTTPS certificates with Route53 and Certbot.	
Development and Operation of BI Tool	Nov 2023 - Nov 2023
• Built the BI tool's server on Flask and NextJS. • Developed API to deliver social relationship data. • Rendered social relationships as a directed graph with Cytoscape and SigmaJS. • Developed API for data labeling. • Built the components in NextJS on the Carbon Design System. • Managed HTTPS certificates with Route53 and Certbot. • Deployed to an EC2 instance (Ubuntu 22.04) with Docker Compose.	

Deployment and Operation of Airflow Service

Jan 2023 - Feb 2024

- Wrote Airflow DAGs whose PythonOperators drive the scraper management server on a schedule.
- Wrote DAGs that spin up AWS EMR through boto3 to clean and load the data.
- Ran the DAGs on the Sequential Executor.
- Managed HTTPS certificates with Route53 and Certbot.
- Deployed to an EC2 instance (Ubuntu 22.04) with Docker Compose.

Development of Data Deduplication Feature

Nov 2023 - Feb 2024

- Worked with an ML research engineer on a deduplication system: an IVFFlat index in Faiss-CPU, scoring distance between embedding vectors.
- Extracted image embedding vectors with Facebook's SSCD model.
- Extracted text embedding vectors with OpenAI's Ada embedding model.
- Cut index-creation overhead by 50% with the PostgreSQL pgvector 0.5.0 extension.
- Created pgvector IVFFlat and HNSW indexes.
- Moved the stored vectors from numpy.array to pgvector::vector.

Data Warehouse Schema Design for Efficient Data Management

Feb 2024 - Sep 2024

- Designed a warehouse in DBDiagram around a single-source-of-truth table, to hold the duplicates arriving from many sources.
- Redesigned the schema around S3 as the data lake, since one RDS serving as both source and warehouse had grown expensive.
- Designed DataSource, DataWarehouse, DataMart, and SSoT schemas, encompassing approximately 50 tables.

AWS Service Management

- Implemented user-specific service access control based on IAM Roles.
- Migrated EC2 x86 instance types to arm64 for cost savings.
- Operated Aurora RDS PostgreSQL with separate Read and Write Instances.
- Chose between managed services and self-run EC2 by comparing what each actually cost.

Eazel Inc.

Oct 2021 - Jun 2022

MLOps Engineer, MLOps

Jung-gu, Seoul

Art Market Analysis PoC

Apr 2022 - Jun 2022

- Ran entity recognition from HuggingFace NLP over artist CV data to analyze and clean it quantitatively.
- Analyzed articles with word histograms.
- Analyzed articles with sentiment analysis.

Instagram Relationship Data Analysis

Feb 2022 - Apr 2022

- Analyzed the relationship graph with Node2Vec, K-means and PCA.
- Built a NextJS / Flask / CytoscapeJS page that renders those relationships as a directed graph.
- Secured Series A funding based on analysis data.

Instagram Relationship Information Scraper Development

Oct 2021 - Jun 2022

- Worked through several approaches to keep collection running without gaps.
- Built a dynamic-page scraper in Selenium.
- Built a scraper that goes through the public API with requests.
- Built a scraper on Instagrapi, the open-source client for Instagram's private API.
- Ported Instagrapi from Python to Swift so the scraper could run on mobile
- Contributed to Open Source project for social relationship data collection: [Instagrapi](#)

Art Artist Information Scraper Development

Oct 2021 - Nov 2021

- Built a static-page scraper with requests.
- Stood up a fast scraping setup, running EC2 instances in parallel through boto3 and multithreading.
- Collected information on 600,000 artists and 3 million artist CV entries.

PostgreSQL-based Collected Data Management

- Split the schema so raw and cleaned data sit apart.

- Cleaned the data in Python with psycopg2

Eazel Inc.

iOS Developer, iOS Team

Jan 2019 - Oct 2021

Gangnam, Seoul

iOS Application Development and Deployment

- Built CI/CD using Fastlane and Github Actions.
- Built Artwork Hanger, an ARKit feature that hangs a piece on the viewer's own wall.
- Implemented Push Notification feature.
- Built the API layer on Alamofire to call the backend.
- Integrated Unity's Google Cardboard VR through a Unity bridge.
- Implemented asset download from AWS S3 for the VR mode.
- Built a download service that handles several transfers at once.
- Adopted ReactorKit for reactive programming and built out the Model / View / Reactor architecture around it.
- Established unit test environment and progressively expanded coverage.
- Wrote the Auto Layout in SnapKit.
- Built the image view on Kingfisher.
- Developed Pinterest-style UICollectionView.

Finple Inc.

Software Engineer, DevOps

Jan 2018 - Dec 2018

Yeongdeungpo, Seoul

iOS Hybrid Application Maintenance

Jan 2018 - Nov 2018

- Maintained an iOS app built on UIWebView and WKWebView, talking to a VueJS web layer over Alert-based messaging.
- Migrated from Objective-C to Swift.
- Developed features corresponding to ActionCodes transmitted via Alerts.

Android Hybrid Application Maintenance

Jan 2018 - Nov 2018

- Maintained an Android app built on Chrome WebView, talking to the same VueJS web layer over Alert-based messaging.
- Maintained in Java.
- Developed features corresponding to ActionCodes transmitted via Alerts.

NodeJS-based Data Scraper Development

Jan 2018 - Nov 2018

- Reused the existing encryption module to implement login.
- Implemented scraping that walks a page's pagination.

Education

Coursera

Feb 2023

Convolutional Neural Networks

Coursera

Nov 2022

Structuring Machine Learning Projects

Coursera

Oct 2022

Improving Deep Neural Networks: Hyperparameter Tuning, Regularization and Optimization

Coursera

Sep 2022

Neural Networks and Deep Learning

Engineer Information Processing

Nov 2017

Human Resources Development Service of Korea

Languages

- English: AL (OPIC)
- Japanese: Conversational
- Korean: Native

Korea Polytechnic University

Mar 2010 - Feb 2018

B.S., Computer Engineering (GPA: 3.4 / 4.5) (Major GPA: 3.73 / 4.5)

Siheung, Gyeonggi-do

- **Relevant Coursework:** Data Structures, Algorithms (C), Operating System, Computer Architecture, Cloud Computing
- [Probono] Furniture Arrangement System Using Augmented Reality for the Socially Vulnerable

Other Activities

Guryong Village Coal Briquette Volunteering	Jan 2018
Participated in the Let us go iOS Hackathon Built a layer that returns OpenAI API responses in a fixed recipe format	Jun 2024
Republic of Korea Army, Sergeant (Professional NCO) Honorable Discharge. Created an XLSX payroll calculator using Visual Basic Macro	Jun 2011 - Sep 2013

HUINNO

Dec 2025 - Present

Data Engineer, MLOps Platform

Dec 2025 - Present
Seoul

Led requirements discovery and greenfield MLOps delivery

- **Led requirements discovery and greenfield MLOps delivery:** Interviewed researchers and examined data, code and execution workflows to identify producer dependencies, scattered datasets and missing versions and lineage. Assigned clear responsibilities to Airflow for execution and retries, dbt for SQL transformations, Iceberg for table versions and DataHub for lineage, then designed and implemented the shared platform.
- **Turned tacit data-production knowledge into shared pipelines:** Data was scattered across nodes, generation code was unversioned and access depended on a few producers, making datasets difficult to obtain in their absence. Replaced SSH-based file transfers with shared storage and Airflow/dbt Bronze/Silver/Gold pipelines with lineage in DataHub. Implemented the common framework, research-data pipelines, MIMIC-IV ingestion and operational ETL.

Built and operated on-premises bare-metal Kubernetes

- **Built and operated on-premises bare-metal Kubernetes:** Sole owner of cluster construction on a two-person platform team. Built a 10-node cluster from company hardware and codified networking, storage, identity and execution environments in 12 Pulumi stacks (as of September 2026). Tracked infrastructure changes and inter-stack dependencies to support repeatable deployment.
- **Led hardware configuration and operations:** Disassembled idle PCs and workstations to reclaim and reinstall RAM and NVMe drives; selected and replaced network equipment. Addressed NUMA and dual-channel configuration, memory pressure and swap, and separated the PostgreSQL data disk, adapting both hardware and system settings to actual workloads.

Designed Iceberg catalog and DuckDB query access

- **Designed Iceberg catalog and DuckDB query access:** Used SeaweedFS S3-compatible storage and its Iceberg REST Catalog to manage Silver/Gold data and metadata with NVMe/HDD storage tiers. Built PostgreSQL + pg_duckdb query access. For dimension-table generation, resolved the latest metadata location from the catalog and passed it to DuckDB, avoiding version inference from filenames.

Built data-version and experiment reproducibility foundations

- **Built data-version and experiment reproducibility foundations:** Models and performance were shared in Notion without linked records of data, generation code and execution environments. Implemented manifest freeze/restore tools and validation procedures for data versions, code commits and environment metadata; deployed MLflow tracking and a model registry with SeaweedFS artifact storage. Research-code integration and end-to-end data-to-model reproducibility validation remain in progress.
- **Established operational checks and ownership boundaries:** Implemented recovery and integrity checks for missing raw data and separated services and buckets by environment. Defined researcher ownership of domain ETL and platform ownership of execution, lineage, validation and retries; refined rollout plans through follow-up interviews. Connected Dex/OIDC identity to Kubernetes RBAC and built a shared KubeRay training environment to support research-code migration.

Eazel Inc.

Jan 2019 - Sep 2024

Integrated Data Platform for Art Investment Analysis

- **Core Problem & Goal:** All art market data, including **artists, artworks, auctions, social networks, institutions, and artist CVs**, was fragmented across multiple sources. The final goal was to **build a reliable, integrated data pipeline** to encompass all this data, utilize it as an AI and statistics-based analysis platform, and **contribute to securing Series B funding**.
- **My Role:** As the Data Engineering Team Lead, I led the entire project, from the data pipeline's **architecture design to its implementation, operation, and the development of an LLM agent prototype**.
- **Problem-Solving Process:**

- * **Trade-off Analysis (Pandas vs. PySpark):** Since the project's final goal was to collect all data, I **designed the architecture assuming** the high probability that the data scale would grow to hundreds of millions of records in the future. To secure future **scalability and reliability** over current development speed, I made the decision to proactively **adopt PySpark**, despite the higher initial overhead.
 - * **Specific Implementation (How):** Implemented the data purification and loading pipeline by having the **Python files that define each data processing stage included and executed sequentially as EMR Steps** when the AWS EMR cluster is instantiated. Within this pipeline, I built a system that utilized Facebook's **SSCD** and OpenAI's **Embedding** models and filtered duplicate data using **PGVector's** HNSW index.
 - **Result & Impact:**
 - * **Business Impact:** The high-quality, integrated data platform that was built became a core company asset and played a **decisive role in successfully securing Series B funding**.
 - * **Technical Achievement:** By proactively adopting PySpark, I established the foundation for a scalable architecture that could stably operate the pipeline without a separate migration, even as data volume surged.
- [go back](#)

Central Management & Monitoring Server for Large-Scale Data Scrapers

- **Core Problem & Goal:** As we operated **over 50 scraper instances simultaneously** for data collection, managing the status of each instance individually consumed significant resources. The goal was to **maximize the stability and operational efficiency of the data collection pipeline** by building a server to centrally control and monitor all scrapers in real-time.
 - **My Role:** I was responsible for the **entire End-to-End development**, from the server's architecture design to API endpoint development, WebSocket communication, and Nginx/HTTPS deployment.
 - **Problem-Solving Process:**
 - * **Technical Consideration (Why):** Judged that a **WebSocket** approach, which maintains a persistent bidirectional channel, was more efficient for real-time monitoring than HTTP Polling, as it reduces server load and propagates status changes instantly.
 - * **Trade-off Analysis (Trade-off):** Considered 'Flask' and 'FastAPI'. As the project's core need was stable real-time communication over complex async logic, I **chose 'Flask'**, which I was more familiar with and had a more mature library ecosystem (Flask-SocketIO), to prioritize development speed and stability.
 - * **Specific Implementation (How):** Developed scraper control APIs based on 'Flask' and modularized the code with 'Flask-Blueprint'. Implemented real-time status/metric exchange via '**WebSocket**'. Secured login credentials via '**SHA-256**' encryption and applied an HTTPS certificate from '**Certbot**' using '**Nginx**' as a reverse proxy.
 - **Result & Impact:**
 - * **Maximized Operational Efficiency:** Dramatically reduced operational resources by enabling central management of over 50 scraper instances via a single API, which were previously managed manually.
 - * **Improved Stability & Reliability:** Significantly improved the data collection pipeline's overall stability and reliability by enabling immediate identification and remote restart of failed instances.
- [go back](#)

Full-Stack BI Tool for Social Network Analysis and Data Purification

- **Core Problem & Goal:** The complex graph data from ML models was difficult for non-developer art experts to analyze. An efficient tool was needed for them to visually explore the network and simultaneously label data to improve its quality.
- **My Role:** **Led the entire Full-Stack development**, from data structure design and API development (Flask) to frontend (Next.js), visualization (Cytoscape/SigmaJS), and final deployment (Docker, Nginx).
- **Problem-Solving Process:**
 - * **Technical Consideration (Why):** The key challenge was smoothly rendering large-scale graphs in a browser. I initially used **Cytoscape.js**, but migrated to the WebGL-based **Sigma.js** to improve performance when the number of nodes and edges grew.
 - * **Trade-off Analysis (Trade-off):** Chose IBM's **Carbon Design System** over more common UI libraries. Despite an initial learning curve due to fewer resources, I judged it would improve long-term productivity with its specialized components for data visualization dashboards.
 - * **Specific Implementation (How):** Developed RESTful APIs for social network and labeling data with **Flask**. Built the frontend with **Next.js** and UI components with **Carbon Design System**. Finally, containerized and deployed the Nginx, Next.js, and Flask servers on an EC2 instance using **Docker Compose**.
- **Result & Impact:**
 - * **Improved Work Efficiency:** Dramatically reduced the time required for analysis by enabling art experts to directly explore social network data through the BI tool, instead of requesting it from developers.

- * **Improved Data Quality:** Provided a feature for experts to directly correct and label data, which contributed to improving the quality of training data for the ML model and created a foundation for a virtuous cycle of data purification.
- [go back](#)

Cost-Effective Airflow Implementation for Data Pipeline Automation

- **Core Problem & Goal:** The manual operation of the data collection and purification pipeline was inefficient and prone to errors. The goal was to build a stable scheduling system to **automate execution, monitoring, and reprocessing** while **minimizing operational costs**.
 - **My Role:** I was responsible for the **entire workflow automation process**, including Airflow infrastructure design, EC2/Docker-based deployment, and authoring core DAGs.
 - **Problem-Solving Process:**
 - * **Trade-off 1 (Infrastructure):** Chose to **deploy Airflow directly on EC2 with Docker Compose**, which required more initial setup effort, to reduce infrastructure costs by over 90% compared to the expensive **AWS Managed Airflow (MWAA)**. This was also driven by MWAA's version constraints which did not support my preferred **TaskFlow API**.
 - * **Trade-off 2 (Executor):** Chose the simplest and most stable **Sequential Executor** to minimize infrastructure complexity, as the project's core need was sequential stability over the parallel processing offered by the more complex **Celery Executor**.
 - * **Specific Implementation (How):** Deployed Airflow on EC2 using **Docker Compose**. Authored a scraping DAG that calls the scraper management server's API via a **PythonOperator**, and a data purification DAG that triggers an EMR job using **boto3**.
 - **Result & Impact:**
 - * **Maximized Operational Efficiency:** **Fully automated** the daily manual data tasks, reducing the time spent on related operations by over 95%.
 - * **Improved Stability & Observability:** Enabled central monitoring of all pipeline statuses and logs via the Airflow UI, significantly reducing the time to identify and respond to failures.
- [go back](#)

Embedding-based Image/Text Deduplication System Development and Optimization

- **Core Problem & Goal:** The initial deduplication system had significant **unnecessary overhead**, as it required downloading embeddings from the DB, converting data types, and creating an index externally. The goal was to **optimize the pipeline** and improve data quality by handling all logic within the database.
 - **My Role:** Analyzed the pipeline overhead of the initial system developed by an ML engineer and **led the design and implementation of a new, optimized architecture based on PGVector**.
 - **Problem-Solving Process:**
 - * **Technical Consideration (Problem with old way):** The initial 'Faiss-CPU'-based system was inefficient as it required pulling data from PostgreSQL and **converting incompatible data types** before creating an index locally.
 - * **Specific Implementation (How):** To solve this overhead, I changed the process to handle all logic inside the DB. I performed a data migration from the existing '**Numpy**' array to the '**pgvector::vector**' type and rebuilt the system by creating '**PGVector**'s '**IVFFlat**' and '**HNSW**' indexes.
 - **Result & Impact:**
 - * **Improved Operational Efficiency & Cost Savings:** By completely eliminating the unnecessary data transfer and conversion steps, I **reduced the overall pipeline overhead by over 50%** and simplified the architecture.
 - * **Improved Data Quality:** The optimized system dramatically improved data consistency and reliability, playing a key role in building a high-quality art-market database.
- [go back](#)

Data Warehouse Redesign for Cost Optimization and Data Reliability

- **Core Problem & Goal:** The existing single RDS-centric architecture suffered from severe **data duplication and unnecessary cost issues**. The goal was to improve data reliability and reduce operational costs by separating storage layers for each data processing stage and introducing an **architecture based on a Single Source of Truth (SSoT)**.
- **My Role:** Analyzed the problems in the data pipeline and **led the redesign of the data warehouse structure** to improve cost-efficiency and data reliability.
- **Problem-Solving Process:**

- * **Trade-off Analysis (RDS vs. S3 Data Lake):** While the existing RDS structure was simple to develop initially, its long-term cost and scalability issues were clear. I made the decision to **introduce the low-cost S3 as a data lake**, despite the initial migration costs, to secure long-term storage cost savings and management flexibility.
- * **Specific Implementation (How):** Visually designed DataSource, Data Warehouse, and Data Mart layers using the DBDiagram tool. **Designed approximately 50 table schemas, including an SSoT table** to centrally manage duplicate data.
- **Result & Impact:**
 - * **Established Foundation for Cost Savings:** By introducing an S3 data lake, I established a foundation that could dramatically **reduce anticipated storage costs** compared to the previous architecture.
 - * **Improved Data Reliability:** Established a structure that can systematically manage data duplication and improve consistency and reliability through the **SSoT table design**.

[go back](#)

MLOps Engineer, MLOps at Eazel Inc.

Oct 2021 - Jun 2022

Jung-gu, Seoul

MLOps-based Instagram Network Analysis and Investment Insight PoC

MLOps Engineer

- **Core Problem & Goal:** To objectify/quantify artist market influence, which previously relied on experts' subjective judgments, using Instagram network data. The final goal was to prove growth potential with data and **succeed in securing Series A funding**.
- **My Role:** As an MLOps Engineer, I led the construction of a **reproducible End-to-End pipeline** for the PoC, from data collection, **data/model versioning using DVC**, processing inputs for various models, to K-Means/PCA analysis and result visualization.
- **Problem-Solving Process:**
 - * **Technical Consideration (Why):** Lacking deep knowledge of the machine learning models themselves at the time, I chose to collaborate with an ML Research Engineer, allowing us to focus on our respective areas of expertise. My colleague handled various graph embedding models like **Node2Vec** and **GAT**, while I focused on the data pipeline and subsequent analysis/visualization.
 - * **Specific Implementation (How):** Introduced **DVC**, which integrates with Git, to track and version the model's input data and artifacts by linking them to Git commit hashes, thereby **ensuring the reproducibility** of experiments. Afterwards, I took the extracted embedding vectors as input and analyzed artist groups using **K-Means clustering** and **PCA**. Finally, I implemented an interactive network visualization dashboard with these analysis results using **Next.js** and **Cytoscape.js**.
- **Result & Impact:**
 - * **Business Impact:** Proved artist growth potential through the analytics from the developed visualization dashboard, making a **decisive contribution to successfully securing Series A funding**.
 - * **Personal & Technical Growth:** Through this collaborative experience, I gained a deep practical understanding of building machine learning pipelines, and it became the decisive catalyst for my career transition to a Data Engineer.

[go back](#)

Data Scraper Development and Enhancement for Bypassing Instagram Bot Detection

- **Core Problem & Goal:** To solve the problem of being unable to continuously collect data due to Instagram's strong **Bot Detection system**. The goal was to develop a robust scraper that could stably bypass bot detection and collect large-scale social relationship data **without interruption**.
- **My Role:** Responsible for researching, prototyping, and implementing the final solution, exploring various technical approaches (**Selenium**, **requests**, **Instagrapi**, etc.) to solve the bot detection problem.
- **Problem-Solving Process:**
 - * **Trade-off Analysis (Server vs. Mobile Environment):** Confirmed that using a Python-based scraper in a server environment was ultimately blocked by bot detection. Based on the hypothesis that the 'server environment' itself was a major factor in detection, I chose a fundamental solution: to **personally migrate the Python library to Swift**, despite the high initial cost, to emulate requests from a mobile environment.
 - * **Specific Implementation (How):** **Personally ported the core logic of 'Instagrapi' from Python to Swift**, developing a data scraper that runs in an iOS mobile environment. During development, I fixed bugs found in the library and **contributed directly to the open-source project**.
- **Result & Impact:**
 - * **Core Problem Solved:** Succeeded in **stably collecting large-scale social relationship data**, which was impossible with previous methods, providing the foundation for the 'MLOps Analysis PoC' project.

- * **Demonstrated Problem-Solving Ability:** Solved a persistent external system limitation through the creative approach of porting a framework to a different language.

[go back](#)

Initial Data Pipeline Construction using PostgreSQL Schema Separation

- **Core Problem & Goal:** Needed to efficiently separate and manage collected raw data and purified data. In a situation where building a separate warehouse was difficult, the goal was to build a practical pipeline that could **preserve data integrity and reliably serve data within a single DB**.
- **My Role:** Developed the data purification logic and **personally designed and implemented an architecture that separates data layers by separating schemas within PostgreSQL**.
- **Problem-Solving Process:**
 - * **Trade-off Analysis (DB Separation vs. Schema Separation):** Physically separating the database had the disadvantage of doubling costs and management points. Instead, I chose the **PostgreSQL schema separation method**, which allows for logical separation with no additional cost, securing practicality and cost-efficiency.
 - * **Specific Implementation (How):** Created `raw_data` and `purified_data` schemas within **PostgreSQL**. Built a pipeline that performs data purification logic using **Python's psycopg2 library** and stores the data in tables within their respective schemas.
- **Result & Impact:**
 - * **Ensured Data Integrity:** Established an environment where raw data is preserved without changes and analysts can safely use only purified data, reducing data management complexity.
 - * **Paved the Way for Growth:** This experience provided a foundational understanding of the need for data warehouses, paving the way for designing more sophisticated data architectures later on.

[go back](#)

iOS Developer, iOS Team

Jan 2019 - Oct 2021

Gangnam, Seoul

iOS Application Development & Deployment

iOS Developer

- **Core Problem & Goal:** To develop a native iOS application for displaying art content, featuring VR capabilities, asset downloading, and AR functionalities.
- **My Role:** Led the development of the app's backend layer, including the API Client. Implemented detail views and view controllers for artworks, artists, and exhibitions. Developed key features including a Google Cardboard VR view via a Unity Bridge, a download service for VR assets, and an AR-based "Artwork Hanger" feature using ARKit.
- **Problem-Solving Process:**
 - * **Technical Consideration (Why):** Faced with upcoming complex features like VR asset downloading and Unity integration, I anticipated that the existing MVC structure would lead to increased complexity and reduced maintainability. The goal was to adopt a reactive architecture for clearer and more predictable state management.
 - * **Trade-off Analysis (Trade-off):** I first considered **implementing the MVVM pattern manually**. However, I judged that this would require significant initial development resources for defining protocols and data binding logic. As an alternative, I reviewed the reactive programming framework, **ReactorKit**. I was impressed that ReactorKit enforces a clear, 3-step unidirectional data flow — **User Input (Action) -> Business Logic Processing (Mutate) -> Applying State Change (Reduce)** — which fundamentally prevents the complexity of manual implementation and makes the code flow very easy to understand. I concluded that despite the initial learning curve, adopting ReactorKit was more advantageous for long-term maintainability and development efficiency.
 - * **Specific Implementation (How):** Re-architected the app based on ReactorKit. Unified the data flow by defining all user inputs as 'Actions' and state changes as 'State', and integrated the download service using an AWS S3 client into the Reactor logic.
- **Result & Impact:**
 - * **Improved Development Efficiency:** After getting used to the framework, **the process of defining logic according to the Action-Mutate-Reduce structure became very easy and clear**. This led to improved development speed for new features and maintained code consistency.
 - * **Enhanced Stability & Maintainability:** The clear structure reduced the occurrence of state management-related bugs, and shortened the time it took for new team members to understand the codebase.